

Bug Report: EPERM on raw TCP socket for MQTT (Paho) from debug app on Samsung Galaxy Note 10 (SM-N970F)

Svensk sammanfattning

****Problem****
via rå TC

****Felmelding****
Feilet opp

****Vad som händer****
• Appens
• Vid "Tr
• Loggar
• Subscr
appen a
• Nätver

****Vad som gjorts****
• Factory
• Develo
• Alla re
viloläge"
• Byte fra

Feilet kva

****Konsekvenser****
Subscrib

Rapporto

ID: 2026-06-07-MQTT-EPERM-SAMSUNG-NOTE10 | Date: 2026-06-07 | Severity: Medium | Status: Open / Workaround in use

****Date:**** 2026-06-07

****Severity:**** Medium (blocks end-to-end MQTT communication testing on physical target device)

****Status:**** Open / Workaround in use

****Reporter:**** User (via Grok-assisted UAT)

****Affected Component:**** `data/mqtt/MqttSender.kt` + transmit flow in `RFIDManagerScreen`

Summary

When the RFID Manager app (Kotlin + Jetpack Compose, debug build from Android Studio) attempts to publish an MQTT message using Eclipse Paho MQTT client over plain TCP to a local broker, the connection fails with:

```
java.net.SocketException: socket failed: EPERM (Operation not permitted)
```

This occurs inside `Socket.createImpl` `Socket.connect` Paho `TCPNetworkModule.start`.

The failure is consistent even after:

- Factory reset of the device
- Fresh login with primary user account
- Developer options enabled
- All documented Samsung battery / data usage / background restrictions lifted for the app
- Running the app via "Run" directly from Android Studio (not just adb install)
- Using direct broker IP instead of `localhost` + `adb reverse`

****Browser on the same phone**** can open a TCP connection to the broker IP:port (receives `ERR\_EMPTY\_RESPONSE`, which is the expected behavior for a raw MQTT broker). This proves the network path from the device is open.

Environment

****Device:****

- Model: Samsung Galaxy Note 10 (SM-N970F/DS, international Exynos variant)
- No SIM card (Wi-Fi only)
- OS: Android (One UI), post factory reset
- Developer options: Enabled
- USB debugging: Enabled and authorized

****App /**

- Package: `com.joakim.rfidmanager`
- Built and launched via Android Studio ("Run app", debug build)

The app
"Transm
publishe

****App /**

- `targetSdk` = 36
- Uses Eclipse Paho MQTT Client (`paho-mqtt-client`)
- Broker address (current): `tcp://192.168.50.128:1883` (PC Wi-Fi IP, Docker Mosquitto)
- Topic format used by app: `rfidmanager/<uid>/telemetry`
- Payload: Simple JSON (see "App payload example" below)

**Test e

- Broker: Docker `eclipse-mosquitto` on host port 1883
- Subscriber: `test\_subscriber\_persist.py` (listens on `rfidmanager/+/telemetry`, persists to SQLite)
- PC IP on same Wi-Fi: 192.168.50.128

Repro

1. On phone: Perform NFC read + write with "Persist after write" enabled.
2. Go to PERSISTED tab observe the item in the list.
3. Press the "Transmit " control on that item.
4. Observe:
 - Success Toast appears ("Transmit sent...").
 - UI status on the row updates to "Transmitted via Sparkplug".
 - Logcat shows "Attempting MQTT...", followed by the EPERM stack trace.
 - No message arrives at the PC subscriber.

Expec

The app should successfully open a TCP socket to the configured broker, connect via Paho, publish the message, and the subscriber on the test PC should receive it on the expected topic and persist it.

Actual Behavior

Socket creation fails at the OS level with `EPERM`. Higher-level app code treats the send as successful (or at least reaches the post-send logging/Toast path because the exception is swallowed inside `MqttSender`).

Key Log Excerpt (Logcat from Android Studio)

```
MqttSender I Attempting MQTT connect/publish to tcp://192.168.50.128:1883 for uid=047B05CA885884
MqttSender E Failed to connect to MQTT
MqttException (0) - java.net.SocketException: socket failed: EPERM (Operation not permitted)
    at org.eclipse.paho.client.mqttv3.internal.ExceptionHelper.createMqttException(...)
    at ...TCPNetworkModule.start(...)
Caused by: java.net.SocketException: socket failed: EPERM (Operation not permitted)
    at java.net.Socket.createImpl(Socket.java:492)
    at java.net.Socket.connect(Socket.java:619)
...
MqttSender E Failed to publish reading
Client is not connected (32104)
...
MqttSender I === SEND COMPLETE for uid=047B05CA885884 type=ReadEscortMemory ===
MqttSender I Broker used: tcp://192.168.50.128:1883
MqttSender I Topic published: rfidmanager/047B05CA885884/telemetry
```

App Payload Example (what the app actually tries to send)

```
{
  "type": "ReadEscortMemory",
  "uid": "047B05CA885884",
  "timestamp": "<epoch ms>",
  "source": "Manual write page 12",
  "sparkplug": true,
  "data": {
    "memoryBank": 3,
    "address": 12,
    "length": 4,
    "payload": "74657374"
  }
}
```

What Has Been Tried (exhaustive)

- Factory reset + fresh primary account login
- Developer options enabled + USB debugging authorized
- All app-specific settings:
- Mobile data & Wi-Fi Background data usage + "Allow data when Data Saver is on" = ON
- Battery "No restrictions" / Unrestricted
- Device Care Battery "Sätt oanvända appar i viloläge" = OFF (via "Gränser för bakgrundsanvändning")
- Developer options "Behåll inte aktiviteter" ("Don't keep activities") = OFF
- Running via "Run" directly from Android Studio (not pure adb sideload)
- Broker address changed from `localhost` + `adb reverse` to direct PC Wi-Fi IP

- Multiple force-stops + clears + re-runs
- Browser on the same phone can reach `http://192.168.50.128:1883` (gets `ERR\_EMPTY\_RESPONSE` — proves TCP reachability from the device)

No further
search.

Evidence That the Network Path Works

- Chrome on the phone to `http://192.168.50.128:1883` `ERR\_EMPTY\_RESPONSE` (Mosquitto accepts the TCP connection but does not speak HTTP).
- PC-side publish (using `paho-mqtt` from the test env) to the same IP + topic succeeds and is received by the subscriber.

PC-Side

When an identical payload (using the real UID `047B05CA885884`) is published from the PC instead of the phone, the subscriber correctly receives it:

```
Received: ReadEscortMemory from topic rfidmanager/047B05CA885884/telemetry  
Persisted to SQLite (Sparkplug-aware).
```

This proves the broker + subscriber + DB + topic + payload format handling all work.

Impact

- Prevents actual end-to-end MQTT transmission testing from the physical target device.
- Local persistence, UI, and transmit *logic* can still be validated.
- Subscriber side can be validated by publishing from the PC.

Workaround

- PC-side publish (via `simulate\_mobile\_publish.py` or direct `paho` script) using real UIDs from phone-persisted posts to exercise the subscriber.
- All transmit-related code paths, logging, UI feedback, and payload construction are exercised and verified on the phone.

Hypothesis

1. Samsung One UI applies additional sandboxing / SELinux / AppOps restrictions on debuggable apps (especially those launched via Android Studio) for low-level socket operations, even when the user has granted "unrestricted data usage".
2. The restriction is specific to Wi-Fi-only operation (no SIM card) combined with high `targetSdk` (36).
3. `adb reverse` vs. direct IP does not change the outcome (tested both).

References / Links (internal wiki)

- `wiki/log.md` — entries from 2026-06-06 and 2026-06-07 (detailed test history)
- `wiki/Fas2-Implementation-Overview.md`
- `wiki/Rollfördelning-och-Arbetsätt.md` (roles used during this UAT)
- App files: `app/src/main/java/com/joakim/rfidmanager/data/mqtt/MqttSender.kt`, `RFIDManagerScreen.kt`

Suggestion

- Reproduce on a stock Pixel device (same Android version) with the exact same debug APK.
- Compare `adb shell dumpsys appops` and SELinux context for the package between a working device and this Samsung.
- Test with `targetSdk` lowered temporarily.
- Test with a release-signed (non-debug) build installed via `adb install`.
- Check whether `android.permission.INTERNET` + the data usage toggles are sufficient, or if a runtime permission / special app access is also required on this firmware.
- Investigate if "Wi-Fi power saving", "Secure Wi-Fi", or Knox-related policies are still active despite user-visible toggles being off.

All information
Project file
append-